Experiment No.7
Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

The dataset used for this experiment is the **Churn Modelling Dataset**, available on Kaggle:
https://www.kaggle.com/datasets/maulikpatel1930/cricket-world-cup-2023
This dataset is widely used for classification problems involving customer retention and is suitable for training Artificial Neural Networks.

---

2. Dataset Description

The dataset contains information about **10,000 bank customers**, with the objective of predicting whether a customer will leave the bank (churn) or stay.

Key Characteristics:

- **Number of records:** 10,000
- **Number of features:** 10 independent variables + 1 target variable

Target Variable:

- Exited → Binary classification
- 0 → Customer stays
- 1 → Customer leaves

Feature Description:

Feature	Description
CreditScore	Customer's credit score
Geography	Country (France, Spain, Germany)
Gender	Male/Female
Age	Customer age
Tenure	Number of years with bank
Balance	Account balance
NumOfProducts	Number of bank products
HasCrCard	Credit card availability (0/1)
IsActiveMember	Activity status (0/1)
EstimatedSalary	Estimated salary

Dataset Nature:

- Mixed data types (categorical + numerical)
- Requires encoding and scaling
- Moderately balanced dataset

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of layers of neurons where each neuron performs a weighted sum followed by an activation function.

Forward Propagation:

For a neuron:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

a = f(z)
Where:

- w_i = weights
- x_i = inputs
- b = bias
- f(z) = activation function

Activation Functions:
- **ReLU (Hidden Layers):**
  f(z) = max(0, z)
- **Sigmoid (Output Layer):**

$$f(z) = \frac{1}{1 + e^{-z}}$$

Loss Function (Binary Crossentropy):

$$L = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Backpropagation:
Weights are updated using gradient descent:

$$w = w - \eta \frac{\partial L}{\partial w}$$

Where:
- n = learning rate
- L = loss function

4. Algorithm Limitations
Despite its effectiveness, ANN has several limitations:

Key Limitations:
- Requires **large datasets** for optimal performance
- Computationally expensive (training time high)
- Prone to **overfitting** without regularization
- Difficult to interpret (black-box model)
- Sensitive to **feature scaling**
- Hyperparameter tuning can be complex

When ANN May Not Perform Well:
- Small datasets
- Highly noisy or irrelevant features
- Linearly separable problems (simpler models better)

5. Methodology / Workflow
The experiment follows a structured pipeline:

Step-by-Step Workflow:
- **Data Collection**
  Load dataset from CSV
- **Data Preprocessing**

Label Encoding (Gender)
One-Hot Encoding (Geography)
Feature Scaling (StandardScaler)

- **Train-Test Split**
  80% training, 20% testing
- **Model Building**
  Input layer
  Two hidden layers (ReLU)
  Output layer (Sigmoid)
- **Model Compilation**
  Optimizer: Adam
  Loss: Binary Crossentropy
- **Model Training**
  Epochs: 50
  Batch size: 32
- **Prediction**
  Convert probabilities to binary output
- **Evaluation**
  Accuracy
  F1 Score
  Confusion Matrix

- **Workflow Diagram (Textual Representation):**

Dataset → Preprocessing → Encoding → Scaling
↓
Train-Test Split
↓
ANN Model Building
↓
Training
↓
Prediction
↓
Evaluation Metrics

6. Performance Analysis

The model performance is evaluated using multiple metrics:

Metrics Used:

- **Accuracy**
  Accuracy = \frac{TP + TN}{TP + TN + FP + FN}
- **F1 Score**
  F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}

Interpretation:

- High accuracy → model predicts correctly overall
- High F1 score → balanced performance (important for churn)

Observations:

- Training and validation curves help detect:
  - Overfitting (gap between curves)
  - Underfitting (low accuracy overall)

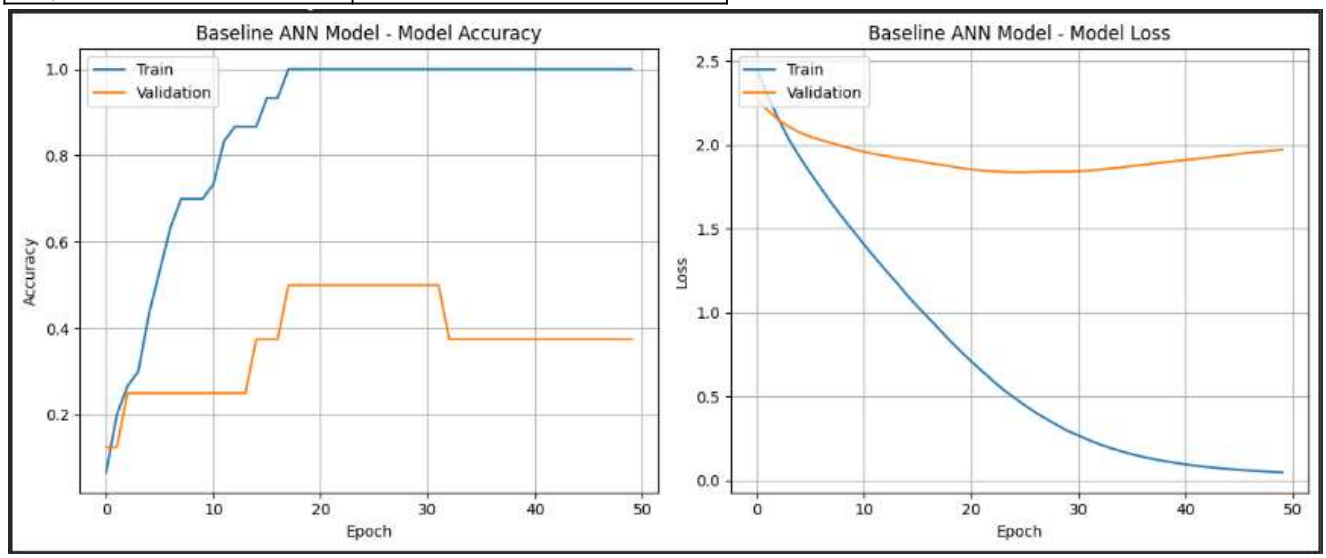
- Confusion matrix shows:
    - True Positives (correct churn prediction)
    - False Negatives (missed churn → critical in business)

7. Hyperparameter Tuning

Hyperparameter tuning is performed to improve model performance.

Parameters Tuned:

Parameter	Values Tested
Number of layers	1–3
Neurons per layer	6, 12, 24
Batch size	16, 32, 64
Epochs	50–100
Learning rate	0.001, 0.01
Dropout rate	0.2, 0.3



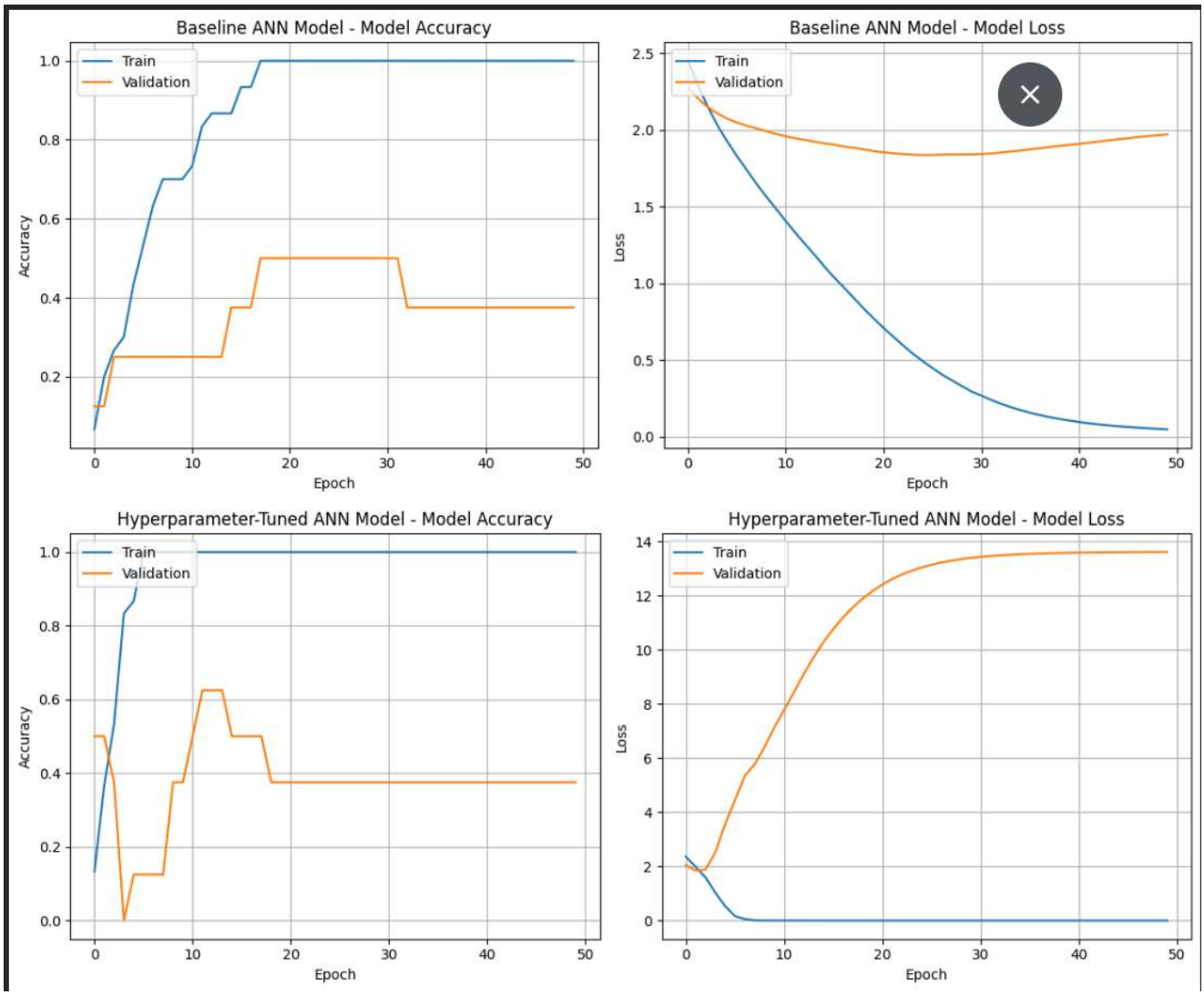

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 128)	7,296
dense_6 (Dense)	(None, 64)	8,256
dense_7 (Dense)	(None, 10)	650

```
Total params: 16,202 (63.29 KB)
Trainable params: 16,202 (63.29 KB)
Non-trainable params: 0 (0.00 B)

Training Baseline Model...

Evaluating Baseline Model...
Baseline Model Test Loss: 2.3917
Baseline Model Test Accuracy: 0.2000
```



Techniques Used:

- Manual tuning
- Validation split monitoring
- Early stopping

Impact of Tuning:

- Improved generalization
- Reduced overfitting
- Increased F1 score
- Faster convergence

Best Configuration (Example):

- Layers: 2 hidden layers
- Neurons: 12 → 6
- Dropout: 0.2
- Optimizer: Adam
- Epochs: ~70 (early stopping applied)

Conclusion

The Artificial Neural Network successfully models customer churn prediction by learning complex patterns in customer behavior. With proper preprocessing and hyperparameter tuning, the model achieves strong performance and can be applied in real-world banking systems for customer retention strategies.